# RSG Mini AI Business Intelligence Assistant

**Complete Project Architecture, Engineering Workflow & Interview Preparation Guide**

GET Selection Technical Assessment | Vectorless RAG Architecture | 24/24 Automated Tests Passed

## 1. Project Overview & Objective

**Goal:** Build an application that allows business users to ask natural language questions about sales transactions and receive verified, deterministic answers backed by real database queries.

> **The Key Architectural Principle:** In financial and business intelligence data, *LLMs must never invent or guess numbers*. We use AI strictly to **classify user intent** and **extract parameters**, while executing **allowlisted, parameterized, deterministic SQLite queries** to calculate exact results.

## 2. Full Technology Stack

- **Backend Gateway:** `FastAPI` (Python 3.10+) — ASGI framework with Pydantic schema validation.
- **Database Engine:** `SQLite3` — ACID-compliant relational database with B-tree indexes on order dates, products, and regions.
- **AI / Interpretation Layer:** Dual-Engine — supports zero-shot LLM classification (Groq/OpenAI/Ollama) and a deterministic regex word-boundary fallback requiring 0 external API keys.
- **Frontend UI:** HTML5, modern CSS3, and `Chart.js` for interactive visual dashboards.
- **Automated Testing:** `Pytest` with 24 automated unit and integration tests.

## 3. Data Lineage & Cleansing Policy (51 Total Rows)

The source CSV ( `data/sales_data.csv` ) contains 51 transactions. Rather than silently deleting invalid rows, all 51 rows are imported into SQLite, tagging rows with `is_eligible` (1 or 0) and an explicit `validation_note` :

- **48 Eligible Transactions:** Completed sales with positive quantities and valid prices.
- **3 Excluded Transactions:**
  - `TXN0033` : Status is **Returned** → Excluded from net revenue.
  - `TXN0034` : Quantity is **0** → Invalid transaction.
  - `TXN0051` : Unit Price is **Missing/NULL** → Incomplete record.
- **Net Revenue Formula:** `Net Revenue = (Quantity × Unit Price) - Discount Amount`

## 4. Core Assessment Questions & Converted SQL Statements

| Question | Intent | Exact Executed SQL Query | Verified Result |
|---|---|---|---|

| | | | |
|---|---|---|---|
| **1. Total revenue?** | `total_revenue` | `SELECT ROUND(COALESCE(SUM(revenue), 0), 2) AS total_revenue, COUNT(*) AS transactions FROM sales_transactions WHERE is_eligible = 1;` | ₹1,439,200.00 (48 txns) |
| **2. Highest revenue product?** | `top_product` | `SELECT product, ROUND(SUM(revenue), 2) AS revenue, SUM(quantity) AS units_sold, COUNT(*) AS transactions FROM sales_transactions WHERE is_eligible = 1 GROUP BY product ORDER BY revenue DESC, product ASC LIMIT 1;` | Monitor (₹473,000.00, 50 units sold) |
| **3. Monthly revenue trend?** | `monthly_trend` | `SELECT strftime('%Y-%m', order_date) AS month, ROUND(SUM(revenue), 2) AS revenue, COUNT(*) AS transactions FROM sales_transactions WHERE is_eligible = 1 GROUP BY month ORDER BY month;` | 6 months (Jan: ₹210.2k, Feb: ₹234.6k, Mar: ₹223.9k, Apr: ₹240.7k, May: ₹318.5k, Jun: ₹211.2k) |
| **4. Highest sales region?** | `top_region` | `SELECT region, ROUND(SUM(revenue), 2) AS revenue, COUNT(*) AS transactions FROM sales_transactions WHERE is_eligible = 1 GROUP BY region ORDER BY revenue DESC, region ASC LIMIT 1;` | South (₹411,200.00 across 13 txns) |
| **5. Revenue change between two months?** | `compare_months` | `SELECT strftime('%Y-%m', order_date) AS month, ROUND(SUM(revenue), 2) AS revenue, COUNT(*) AS transactions FROM sales_transactions WHERE is_eligible = 1 AND strftime('%Y-%m', order_date) IN ('2026-01', '2026-02') GROUP BY month ORDER BY month;` | Increased by ₹24,350.00 (+11.58%) from Jan to Feb 2026 |

## 5. AI Interview Topics Deep-Dive

### A. Generative AI vs. Agentic AI

- **Generative AI:** Single-turn, reactive prompt-to-response generation. Cannot use tools or execute SQL. Predicts next tokens probabilistically, which causes math hallucinations.
- **Agentic AI:** Multi-step, goal-directed autonomous execution with tool-calling loops. The agent evaluates what is needed, invokes tools (e.g. SQLite database queries), validates the outputs, and presents verified figures.
- **Andrew Ng's 4 Agentic Patterns:**
    1. **Tool Use:** Connecting LLMs to deterministic execution tools (our SQLite queries).
    2. **Planning:** Breaking complex instructions into sub-steps.
    3. **Reflection:** Self-checking outputs before presentation.
    4. **Multi-Agent Collaboration:** Specialized agents cooperating on a task.

### B. RAG vs. Vectorless RAG

> **Why Traditional Vector RAG Fails for BI:**
> Vector RAG splits text into chunks, generates vector embeddings, and performs cosine similarity search. When asked *"What is total revenue?"*, Vector RAG retrieves only top-5 arbitrary text chunks. The LLM then sums those 5 chunks and hallucinates an incomplete answer.

**Vectorless RAG (Our Approach):** Queries structured relational databases directly using schema-grounded, allowlisted SQL. It aggregates 100% of rows deterministically with zero math hallucination and millisecond response time.

### C. Agentic Frameworks (LangChain, Google ADK, Dify)

- **LangChain / LangGraph:** Implements state machine graphs. Our assistant maps to a `StateGraph` with a Router Node, a Database Tool Node, and a Formatter Node.
- **Google ADK (Agent Development Kit):** Leverages native Gemini function calling ( `tools=[total_revenue, compare_months]` ) with OpenAPI typed schemas.
- **Dify:** Visual LLMOps platform where our FastAPI endpoints can be imported as a Custom Tool Provider in a drag-and-drop workflow DAG.

## 6. Security Guardrails & SQL Injection Prevention

1. **No Dynamic SQL Generation:** User input is never concatenated into SQL strings.
2. **AST Query Validator ( `validate_sql_query` ):** Restricts queries to single-statement read-only `SELECT` and `WITH` operations. Blocks `DROP` , `DELETE` , `UPDATE` , `INSERT` , `ALTER` , and multi-statement execution.
3. **Parameterized Queries:** Uses `?` parameter binding for safe variable interpolation.

## 7. How to Run the Application

### Command Line Interface (CLI):

```
python ask.py "What is the total revenue?"
```

### Web Dashboard:

```
source .venv/bin/activate
python -m uvicorn app.main:app --reload
# Open browser at: http://localhost:8000
```

### Automated Tests:

```
pytest -v
```

## 8. Top Interview Questions & Model Answers

**Q1: Why didn't you let the LLM generate arbitrary SQL directly?**

"Allowing unconstrained Text-to-SQL introduces severe security vulnerabilities (prompt injection and destructive SQL) and non-deterministic hallucinations (miscalculating filters, missing returned items, or inventing tables). Using Vectorless RAG allows AI to classify intent and extract parameters safely, while deterministic, allowlisted SQL guarantees 100% arithmetic correctness."

**Q2: How did you handle bad or missing rows in the CSV? Did you delete them?**

"Deleting bad rows during ETL destroys data lineage and prevents audit reconciliation. Instead, we preserved all 51 rows in SQLite, adding an `is_eligible` flag and `validation_note`. Only 48 valid rows enter revenue calculations. The 3 excluded rows (TXN0033: Returned, TXN0034: Qty 0, TXN0051: Missing Price) remain stored and can be inspected via our audit API."

**Q3: How would you scale this application to 50 million transactions?**

"In an enterprise production setting, I would: 1. Migrate from SQLite to a columnar analytics warehouse like DuckDB or ClickHouse. 2. Create materialized rollup views to pre-aggregate daily and monthly revenue. 3. Add a Redis caching layer for common KPI queries with TTL invalidation. 4. Implement dynamic schema tool-calling using LangGraph or Google ADK."

**Q4: What real problems did you encounter during development and how did you debug them?**

"When asking the assessment's verbatim Question 5 ('How did revenue change between two months?'), no month names were specified, so the parser fell through to total revenue. I debugged this by adding a comparison fallback that defaults to comparing the first two recorded months with a note. Additionally, naive keyword matching on SQL injection input like `DROP TABLE sales_transactions;` matched 'sales'. I resolved this using regex word boundaries `\b` and an upfront DDL blocklist."